

Lembranças

Amaru está comprando lembranças em uma loja estrangeira. Existem N **tipos** de lembranças. Há infinitas lembranças de cada tipo disponíveis na loja.

Cada tipo de lembrança tem um preço fixo. Ou seja, uma lembrança do tipo i ($0 \leq i < N$) tem um preço de $P[i]$ moedas, onde $P[i]$ é um inteiro positivo.

Amaru sabe que os tipos de lembranças são classificados em ordem decrescente de preço, e que os preços das lembranças são diferentes. Especificamente, $P[0] > P[1] > \dots > P[N-1] > 0$. Além disso, ele foi capaz de descobrir o valor de $P[0]$. Infelizmente, Amaru não tem outras informações sobre os preços das lembranças.

Para comprar algumas lembranças, Amaru realizará uma série de transações com o vendedor.

Cada transação consiste nas seguintes etapas:

1. Amaru entrega um número (positivo) de moedas ao vendedor.
2. O vendedor coloca essas moedas em uma pilha sobre a mesa na sala dos fundos, onde Amaru não pode vê-las.
3. O vendedor considera cada tipo de lembrança $0, 1, \dots, N-1$ nessa ordem, um por um. Cada tipo é considerado **exatamente uma vez** por transação.
 - Ao considerar o tipo de lembrança i , se o número atual de moedas na pilha for pelo menos $P[i]$, então
 - o vendedor remove $P[i]$ moedas da pilha, e
 - o vendedor coloca uma lembrança do tipo i na mesa.
4. O vendedor dá a Amaru todas as moedas restantes na pilha e todas as lembranças sobre a mesa.

Observe que não há moedas ou lembranças na mesa antes do início da transação.

Sua tarefa é instruir Amaru a realizar um certo número de transações, para que:

- em cada transação ele compre **pelo menos uma** lembrança, e
- no total ele compra **exatamente** i lembranças do tipo i , para cada i tal que $0 \leq i < N$.
Observe que isso significa que Amaru não deve comprar nenhuma lembrança do tipo 0.

Amaru não precisa minimizar o número de transações e possui uma quantidade ilimitada de moedas.

Detalhes de implementação

Você deve implementar o seguinte procedimento.

```
void buy_souvenirs(int N, long long P0)
```

- N : o número de tipos de lembranças.
- $P0$: o valor de $P[0]$.
- Este procedimento é chamado exatamente uma vez para cada caso de teste.

O procedimento acima pode fazer chamadas para o seguinte procedimento para instruir Amaru a realizar uma transação:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : o número de moedas entregues ao vendedor por Amaru.
- O procedimento retorna um *pair*. O primeiro elemento do *pair* é um vetor L , contendo os tipos de lembranças que foram comprados (em ordem crescente). O segundo elemento é um inteiro R , o número de moedas devolvidas ao Amaru após a transação.
- É necessário que $P[0] > M \geq P[N - 1]$. A condição $P[0] > M$ garante que Amaru não compre nenhuma lembrança do tipo 0, e $M \geq P[N - 1]$ garante que Amaru compre pelo menos uma lembrança. Se essas condições não forem atendidas, sua solução receberá o veredito `Output isn't correct: Invalid argument`. Observe que, ao contrário de $P[0]$, o valor de $P[N - 1]$ não é fornecido na entrada.
- O procedimento pode ser chamado no máximo 5000 vezes em cada caso de teste.

O comportamento do corretor **não é adaptativo**. Isso significa que a sequência de preços P é fixada antes de `buy_souvenirs` ser chamado.

Restrições

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ para cada i tal que $0 \leq i < N$.
- $P[i] > P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.

Subtarefas

Subtarefa	Pontuação	Restrições adicionais
1	4	$N = 2$
2	3	$P[i] = N - i$ para cada i tal que $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ para cada i tal que $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ para cada i tal que $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.
6	33	Sem restrições adicionais.

Exemplo

Considere a seguinte chamada.

```
buy_souvenirs(3, 4)
```

Existem $N = 3$ tipos de lembranças e $P[0] = 4$. Observe que existem apenas três sequências possíveis de preços P : $[4, 3, 2]$, $[4, 3, 1]$, e $[4, 2, 1]$.

Suponha que `buy_souvenirs` chama `transaction(2)`. Suponha que a chamada retorne $([2], 1)$, significando que Amaru comprou uma lembrança do tipo 2 e o vendedor lhe devolveu 1 moeda. Observe que isso nos permite deduzir que $P = [4, 3, 1]$, já que:

- Para $P = [4, 3, 2]$, `transaction(2)` teria retornado $([2], 0)$.
- Para $P = [4, 2, 1]$, `transaction(2)` teria retornado $([1], 0)$.

Então `buy_souvenirs` pode chamar `transaction(3)`, que retorna $([1], 0)$, significando que Amaru comprou uma lembrança do tipo 1 e o vendedor lhe devolveu 0 moedas. Até agora, no total, ele comprou uma lembrança do tipo 1 e uma lembrança do tipo 2.

Finalmente, `buy_souvenirs` pode chamar `transaction(1)`, que retorna $([2], 0)$, significando que Amaru comprou uma lembrança do tipo 2. Observe que também poderíamos ter usado `transaction(2)` aqui. Neste ponto, no total Amaru tem uma lembrança do tipo 1 e duas lembranças do tipo 2, conforme necessário.

Corretor de Exemplo

Formato de entrada:

N

$P[0] \ P[1] \ \dots \ P[N-1]$

Formato de saída:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Aqui $Q[i]$ é o número de lembranças do tipo i compradas no total para cada i tal que $0 \leq i < N$.